

Task 3 — Service Discovery

Part A — Service Discovery

Implement:

- Service registration
- Service discovery
- Eureka
- Service-to-service lookup

Part B — API Gateway

Build an API Gateway and configure:

- Routing
- Service forwarding
- Request filtering
- Authentication preparation
- Error handling

Step 1 — Create Eureka Server

In **Spring Initializr**, create a new project.

Use:

Name: service-registry

Artifact: service-registry

Language: Java

Java: 17

Packaging: Jar

Add the dependency:

Eureka Server

Then click **Generate**, extract the ZIP, and import it into Eclipse.

Step 2 — Enable Eureka Server

Open

src/main/java

→ com.example.serviceregistry

→ ServiceRegistryApplication.java

Replace it with:

```
package com.example.serviceregistry;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
```

```
@SpringBootApplication
```

```
@EnableEurekaServer
```

```
public class ServiceRegistryApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(ServiceRegistryApplication.class, args);
```

```
    }
```

```
}
```

The important line is:

```
@EnableEurekaServer
```

This makes the application an **Eureka Server**.

Step 3 — Configure Eureka

Open:

```
Src/main/resources/application.yml
```

Put:

```
spring:
```

```
  application:
```

```
    name: service-registry
```

```
server:
```

```
  port: 8761
```

```
eureka:
```

```
  client:
```

```
    register-with-eureka: false
```

```
    fetch-registry: false
```

Save it.

Your Eureka server will use:

<http://localhost:8761>

Step 4 — Add Eureka Client to User Service

Open your **User Service** pom.xml.

Add:

```
<dependency>  
  
    <groupId>org.springframework.cloud</groupId>  
  
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>  
  
</dependency>
```

Then save the pom.xml.

In Eclipse, Maven should automatically download the dependency.

en:

User Service

→ src/main/resources

→ application.yml

Add

spring:

 application:

 name: user-service

server:

 port: 8081

eureka:

 client:

 service-url:

 defaultZone: http://localhost:8761/eureka/

Step 5 — Configure User Service

Open

User Service

→ src/main/resources

→ application.yml

Add

spring:

 application:

 name: user-service

server:

 port: 8081

eureka:

 client:

 service-url:

 defaultZone: <http://localhost:8761/eureka/>

The important part is:

name: user-service

Eureka will register for the service with this name.

Step 6 — Add Product Service

In **Product Service**, add the same Eureka Client dependency:

```
<dependency>

  <groupId>org.springframework.cloud</groupId>

  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>

</dependency>
```

Then configure:

spring:

application:

name: product-service

server:

port: 8082

eureka:

client:

service-url:

defaultZone: <http://localhost:8761/eureka/>

Step 7 — Add Order Service

In **Order Service**, add:

```
<dependency>

  <groupId>org.springframework.cloud</groupId>

  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>

</dependency>
```

Configure

spring:

application:

name: order-service

server:

port: 8083

eureka:

client:

service-url:

defaultZone: <http://localhost:8761/eureka/>

Step 8 — Start the applications

Start them in Eclipse in this order:

1. Service Registry
2. User Service
3. Product Service
4. Order Service

Then open your browser:

<http://localhost:8761>

You should see something like,

Instances currently registered with Eureka

ORDER-SERVICE

PRODUCT-SERVICE

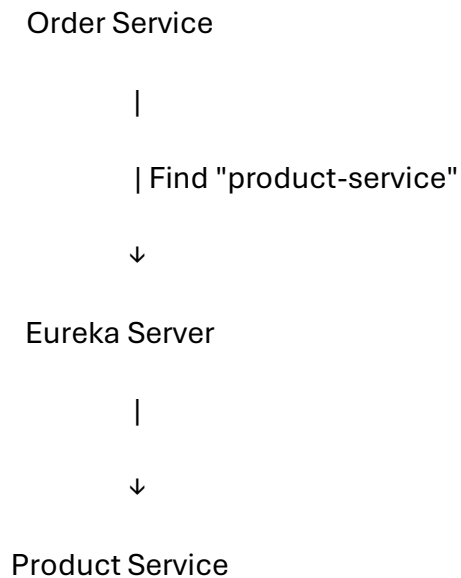
USER-SERVICE

This completes **Service Registration**.

Step 9 — Service-to-Service Lookup

Now let's make **Order Service** communicate with **Product Service** using the Eureka service name.

For example:



use the service name:

<http://product-service>

For Spring Boot, we can configure a load-balanced `RestClient` or use **OpenFeign** for cleaner service-to-service communication.

For this task, I recommend **OpenFeign**.

Step 10 — Add OpenFeign

In **Order Service** → **pom.xml**, add:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```


Then open:

OrderServiceApplication.java

Add:

@EnableFeignClients

Example:

@SpringBootApplication

@EnableFeignClients

```
public class OrderServiceApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(OrderServiceApplication.class, args);
```

```
    }
```

```
}
```

Import:

```
import org.springframework.cloud.openfeign.EnableFeignClients;
```

Step 11 — Create Product Client

Inside Order Service create package:

client

Then create:

ProductClient.java

Use:

```
package com.example.orderservice.client;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping("/api/products/{id}")
    Object getProduct(@PathVariable Long id);
}
```

The important part is:

```
@FeignClient(name = "product-service")
```

Order Service asks Eureka for the location of **product service** instead of hard-coding port 8082.

Step 12 — Final architecture

After Task 3:

